COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	7/22/2026	**DATE SUBMITTED:**	7/22/2026
NAME:	Kherwin Emmanuel D. Milan	**INSTRUCTOR:**	Jimlord M. Quejado

Seatwork 3.1 **Creating a Singly Linked List**

1. PROCEDURE OUTPUT

What procedures did you perform?

```
#include <iostream>
#include "singly_ll.h"


int main() {

    //Creation of nodes KHEM

    //Creating a node stored in the stack
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    //Allocate memory in the head
    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;

    //Storing a value inside the node
    head->data = 'K';
    second->data = 'H';
    third->data = 'E';
    fourth->data = 'M';


    //Linking the nodes:
```

```
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = nullptr;

    std::cout << "Testing of Traversal: \n";
    ListTraversal(head);


    std::cout << "Testing of sllinsertHead: \n";
    sllinsertHead('M', &head);
    ListTraversal(head);

    std::cout << "Testing of sllGeneralInsert: \n";
    sllGeneralInsert('X', head->next->next);
    ListTraversal(head);


    std::cout << "Testing of sllinsertEnd: \n";
    sllinsertEnd('U', &head);
    ListTraversal(head);

    std::cout << "Testing of deleting the node: \n";
    slldeleteNode('U', &head);
    ListTraversal(head);
    slldeleteNode('M', &head);
    ListTraversal(head);
    slldeleteNode('X', &head);
    ListTraversal(head);

    std::cout << "Testing of deleting all nodes: \n";
    sllDeleteList(&head);
    ListTraversal(head);

    return 0;
}
```

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

//Node creation
template <typename T>
class SingleList{
    public:
    T data; //This contains the data to be stored in the node
    SingleList<T>* next = nullptr;
};

template <typename T>
void ListTraversal(SingleList<T>* head){
    if(head == nullptr) {
        std::cout << "The list is empty." << std::endl;
        return;
    }
    while (head != nullptr) {
        //Prints the data of n
        std::cout << head->data;

        if(head->next != nullptr){
            std::cout<<"->";
        }

        //Go to the next node n := next
        head = head->next;
    }

    std::cout << std::endl;

}


template <typename T>
void sllinsertHead(T newData, SingleList<T>** currentHead) {

    //1. Allocate memory for the new node
```

```
    SingleList<T>* newNode = new SingleList<T>;

    //2. Put our data into new node
    newNode->data = newData;

    //3. Set next of the new node to point to  the previous head
    newNode->next = *currentHead;

    //4. Reset Head to point to the new node
    *currentHead = newNode;
}

template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* preNode){

    //1. Check if it is the head node (previous node is null)
    if(preNode ==nullptr){

        //2. If null, print "Previous node cannot be null."
        std::cout<<"Previous value cannot be null"<<std::endl;
        return;
    }

//3. Allocate a new node
SingleList<T>* newNode = new SingleList<T>;

//4. Store data in the new node
newNode->data = newData;

//5. Point new node to the node previous node was pointing to
newNode->next = preNode->next;

//6. Point previous node to the new node
preNode->next = newNode;
```

```
}

template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){

    //1. Allocate new node
    SingleList<T>* newNode = new SingleList<T>;

    //2. Dereference to the head node
    SingleList<T>* currentNode = *head;

    //3. Store data in new node
    newNode->data = newData;

    //4. Point next of new node to NULL
    newNode->next = nullptr;

    //5. Traverse the list until next of the node is null
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    //6. Point the next of the current node to the new node
    currentNode->next = newNode;
}


template <typename T>
void slldeleteNode(T findData, SingleList<T>** head){

    //if there is nothing to be delete?
    if(*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    //Searching the node: 1. the node doesnt exist + we found data
```

```
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }


    //if data not found currNode == nullptr in the while loop
    if(currNode == nullptr) {

        std::cout<<"The data is not found  \n"<<std::endl;
        return;
    }


    //if data was found:
    if(prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }

    delete currNode;
}


template <typename T>
void sllDeleteList(SingleList<T>** head){
    SingleList<T>* current = *head;

    while(current != nullptr){

        SingleList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
```

```
}


#endif
```

```
Testing of sllinsertEnd:
M->K->H->X->E->M->U
Testing of deleting the node:
M->K->H->X->E->M
K->H->X->E->M
K->H->E->M
Testing of deleting all nodes:
The list is empty.
PS C:\Users\kemil\OneDrive\Desktop\DSA>
```

Conclusion:
In this activity, I learned how a singly linked list works in C++. I learned how to create nodes, store data, connect each node using pointers, and display the list through traversal. I also learned how to insert a new node at the beginning, middle, and end of the linked list. I also learned how to delete a specific node and delete all the nodes to free the allocated memory.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *grammar correction and proper punctuation*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used grammarly only to correct grammar, spelling, and punctuation. It did not replace my original thinking, analysis, authorship, or completion of the required work

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.